

Aula 007 - Expressões, Atribuição e Controle

Slide 4 — O que cada linha imprime?

1. Python - `print(-2 ** 2, 2 ** 3 ** 2)`

Imprime -4 512. A potência vem antes do sinal unário $-(2^{**2})$; `**` associa à direita ($2^{**(3^{**2})}$).

2. Java - `println(1 + 2 + "3" + 4 + 5)`

Imprime 3345. Primeiro $1+2=3$; então $3+"3"$ vira "33"; os demais `+` concatenam 4 e 5.

3. JavaScript - `console.log("5" - 2, "5" + 2)`

Imprime 3 52. `-` converte a string em número; `+` concatena.

4. C - `printf("%d\n", 3 > 2 > 1)`

Imprime 0. $(3>2)$ resulta em 1; $1>1$ é falso, representado por zero.

5. Python - `x=0; print(x or "padrão", 3 > 2 > 1)`

Imprime padrão True. `or` devolve seu segundo operando porque zero é falso; a comparação encadeada equivale a $(3>2)$ and $(2>1)$.

6. JavaScript - `console.log(0 == "", "" == "0", 0 == "0")`

Imprime true false true. `==` faz coerção e essa igualdade não é transitiva. `===` não converte tipo.

Slide 5 — Expressões aritméticas

Com $a=3$, $b=4$, $c=5$: $(a+b)*c = 35$; $a+(b*c) = 23$; e $a+b*c = 23$ sob a precedência usual (`*` antes de `+`).

Slide 6 — Precedência e associatividade na prática

As regras variam entre linguagens. Parênteses são úteis com sinais, potências, restos e concatenação.

Slide 7 — Ordem dos operandos e efeitos colaterais

- **Java:** a ordem garantida lê o valor antigo 5 antes da chamada; $5+3=8$, e a atribuição final grava 8. Imprime $a = 8$.
- **C:** o slide relata 20 no GCC 14, mas não é resultado portátil. Leitura e modificação de a na expressão têm ordem insegura; o comportamento pode ser indefinido conforme o padrão C.
- **Go:** o slide ressalta que não se deve presumir uma ordem universal entre essa leitura e a chamada que modifica a .

Slide 8 — Operadores sobrecarregados

Em Java, `'a' + 1` resulta em 98 (promoção numérica do caractere); `"" + 'a' + 1` resulta em "a1" (concatenação). Para `BigDecimal`, o exemplo usa `add` e `multiply`, pois Java não oferece sobrecarga de `+` definida pelo usuário.

Em Python, `__add__` e `__mul__` fazem `Vetor(1,2)+Vetor(3,4)` calcular coordenadas (4,6) e `v*3` calcular (12,18). Ressalva: a classe do slide não define `__repr__`/`__str__`; então `print(v*3)` não imprime literalmente `Vetor(12, 18)`, mas a representação padrão do objeto.

Slide 9 — Conversões de tipos

- (int)3.99 resulta em 3, pois descarta a fração; (byte)200 resulta em -56 em Java.
- byte b=10, c=20; b+c é promovido a int; atribuir diretamente a byte não compila sem cast.
- 1/2 com inteiros resulta em 0; 1/2.0 resulta em 0.5.

Slide 10 — Coerção implícita

JavaScript: "5"-2 → 3; "5"+2 → "52"; "5"*2 → 10; true+1 → 2; []+{} → "[object Object]" no contexto mostrado; null+1 → 1; undefined+1 → NaN; "b"+"a"+ "a"+"a" → "baNaNa".

PHP: "0e123" == "0e456" resulta em true no exemplo, pois a comparação frouxa interpreta as strings numéricas como zero; com ===, resulta em false. Hashes MD5 que começam com 0e seguido apenas de algarismos podem sofrer o mesmo problema ao usar ==.

Slide 11 — Expressões relacionais e booleanas

Com a=3, b=2, c=1, em C a>b>c resulta em 0: (3>2) vale 1, e 1>1 é falso. (a>b)&&(b>c) resulta em 1. Em C, if(-1) é verdadeiro porque inteiro não nulo conta como verdadeiro. Em Python, a>b>c resulta em True, pois a comparação é encadeada.

Slide 12 — Avaliação em curto-circuito

- **Java:** i < lista.length && lista[i] != chave verifica primeiro o limite; o acesso à lista não ocorre se i estiver fora.
- **JavaScript:** u.endereco?.cidade resulta em undefined se endereco é null, sem erro.

Slide 13 — Sentenças de atribuição

Em C, sum=count=0 zera as duas variáveis; atribuição pode ser expressão, então if(x=y) pode atribuir y a x e testar o valor — possível erro por ==. while((ch=getchar())!=EOF) é um uso intencional. Java e C# permitem atribuição como expressão, mas a condição de if precisa ser booleana. Python não permite atribuição comum como expressão, mas tem := em contextos permitidos; a,b=b,a troca valores sem temporária. Go não usa atribuição nem x++ como expressão. Em Rust, atribuição resulta em () e não serve como condição. O slide cita ainda val/let em linguagens funcionais.

Slide 14 — Atribuição de modo misto

- **Java:** double d=3 compila por alargamento; int i=3.7 sem cast não compila; (int)3.7 vale 3; long l=i compila. Com i=3, i += 3.7 compila por conversão implícita da atribuição composta e imprime 6.
- **C:** int i=3.7 resulta em 3. char c=300 pode resultar em 44 numa plataforma comum com char de 8 bits, mas depende da implementação. unsigned int u=-1 vale 4294967295 se o tipo tiver 32 bits.

Slide 15 — Estruturas de controle

Sequência executa em ordem ($a=1$, depois $b=a+2$, logo $b=3$); seleção escolhe um caminho com `if/else`; iteração repete enquanto uma condição vale. `while(x>0)` $x--$ decrementa até x não ser positivo.

Slide 16 — Seleção de dois caminhos: “else pendente”

Com `sum=0`, `count=5`, `result=-1`, o primeiro `if` é verdadeiro e o teste interno `count==0` é falso. Em C, o `else` se liga ao `if` mais próximo sem `else`, portanto executa `result=1`; a saída é 1.

Slide 17 — Seleção múltipla: do switch ao match

- C**: com `index=1`, a execução cai pelos casos sem `break`: incrementa `odd` em `case 3`, incrementa `even` em `case 4` e chega ao `default`, que imprime `Error`.
- Java 14+**: o `switch` como expressão devolve “fim de semana” para `SAB/DOM` e “dia útil” para os dias listados. -> evita a queda entre casos; o exemplo cobre o `enum`.
- Rust**: `match n` devolve “negativo” para negativos, “zero” para 0, “um dígito” para 1 a 9 e “vários dígitos” para os demais. `_` cobre o restante; sem cobertura exaustiva, não compila.

Slide 18 — Laços controlados por contador

- C**: no `for` de 0 a menos que 10, o corpo imprime `i` e soma 2; a etapa soma mais 1. Saída: 0 3 6 9.
- Python**: `for i in range(0,10)` imprime 0 1 2 3 4 5 6 7 8 9; alterar `i` no corpo não altera o iterador `range`.

Slide 19 — Laços lógicos, saídas e iteradores

- Java**: `break` externo imprime os índices do primeiro elemento negativo e sai dos dois laços. Sem a matriz, as coordenadas exatas são indeterminadas.
- JavaScript for...in**: percorre índices como strings (“0”, “1”, “2”); somados a `total=0`, resultam na string “0012”.
- JavaScript for...of**: percorre valores 10,20,30 e imprime 60.
- Zune**: com `dias=366` num ano bissexto, `dias>365` é verdadeiro, mas `dias>366` é falso; nenhuma variável muda e o laço fica infinito. Trate esse caso ou garanta progresso em todo caminho.

Slide 20 — Desvio incondicional: goto

O primeiro `goto fail` depende do `if` que verifica o hash. O `goto fail` seguinte não está entre chaves e é incondicional; a verificação da assinatura nunca é executada. Se o hash falha, o erro é diferente de zero. Se passa, o desvio duplicado ainda salta para `fail`, erro permanece zero e a função retorna 0, aceitando o certificado sem verificar a assinatura.

Slide 21 — Comandos protegidos de Dijkstra

No exemplo do máximo, para $x > y$ só $x \geq y$ é verdadeiro; para $y > x$, só $y \geq x$; se forem iguais, ambas são verdadeiras, mas atribuem o mesmo máximo. Entre guardas verdadeiras, a escolha pode ser não determinística. No exemplo Go, ambos os canais estão prontos; select escolhe uma operação pronta. map[a:505 b:495] após 1000 execuções é apenas um resultado possível, não fixo.